



**Amrita School of Artificial
Intelligence**

COIMBATORE CAMPUS

AI IN MATERIAL SCIENCE

The Afternoon, Step by Step

What we do, what you should see, and what to try yourself

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of Artificial Intelligence

SESSION

14:30 to 17:00

NOTEBOOKS

Two, in notebooks/

DATA

Bundled. Nothing to download

KEEP THIS

The numbers below tell you if you are on track

OFFERING

University of Calicut, 24 September 2026

How to use this sheet

Keep it beside your laptop. Each step names the notebook section, says what should appear on your screen, and gives you something to try.

The numbers in the “you should see” boxes are what a correct run produces. If yours differ by a lot, something went wrong earlier, so say so rather than continuing.

We move together. Do not run ahead: the interesting part of each step is the discussion before it.

Part 1. What AI does in chemistry and materials

14:30, about thirty minutes. Nothing to type.

A survey of the field, so the code afterwards has somewhere to sit. Four ideas carry the whole afternoon:

1. A molecule has to become a **vector** before any model can touch it, and the choice of vector matters more than the choice of model.
2. Once it is a vector, **predicting a property is a matrix equation**, which you solved this morning.
3. Running the map **backwards**, from a property you want to a structure to make, is what people now call inverse design.
4. An **agent** is the same model with tools and a loop around it.

Questions are welcome during this part, not only at the end.

Part 2. Hands-on 1: molecules in Python

15:00, about fifty minutes. Open `notebooks/01-molecules-in-python.ipynb` and run cells from the top with **Shift+Enter**.

Step 1. A molecule is a line of text (sections 1 and 2)

You read a SMILES string with RDKit, draw five molecules, and turn one into four numbers.

NOTE

You should see a grid of five structures, then a table of descriptors. Paracetamol should read `MolWt 151.165`, `LogP 1.351`, `TPSA 49.33`, `NumRings 1`.

Try it: change one of the SMILES strings to a molecule you know, and run the cell again. If RDKit prints `None`, the string is not valid, which is itself worth seeing.

Step 2. The dataset (section 3)

1128 molecules whose solubility in water was measured in a laboratory.

NOTE

You should see `(1128, 4)` and then `molecules with descriptors: 1128`, followed by two plots. The left plot should slope downward: greasy molecules dissolve badly.

Step 3. The fit, with the pseudoinverse (section 4)

This is this morning's mathematics on a real chemical problem. Build the matrix `A`, one row per molecule, and solve `w = pinv(A) y`.

NOTE

You should see five weights, and `R2 = 0.777`. The weight on `LogP` should be about `-0.885`, and it is negative because greasier means less soluble.

Notice: there was no training loop, no learning rate, and no waiting. One solve.

Step 4. Honest evaluation (section 5)

Fit on 80 percent of the molecules, score on the 20 percent the model never saw.

NOTE

You should see `4 descriptors, test R2 = 0.812`.

Step 5. Better features, and a spectacular failure (section 6)

Add a 512-bit fingerprint, which records which fragments the molecule contains. Same solve, more columns.

PITFALL

You should see a training R2 of 0.948 and a test R2 of about minus 97 million billion. That is not a typo and your laptop is not broken. This is the most useful thing in the notebook, so stop here and look at it.

Step 6. Why it broke (section 6b)

Look at the singular values of the matrix. The largest is about 7050, the smallest about 1.16×10^{-16} , so the condition number is around 6×10^{19} .

The pseudoinverse inverts each singular value. Dividing by 1×10^{-16} multiplies the noise in that direction by 1×10^{16} , and the fit uses those directions to pass exactly through the training molecules.

The fix is one argument: `np.linalg.pinv(A, rcond=1e-3)` throws those directions away.

NOTE

You should see the sweep, then `descriptors + fp, test R2 = 0.863`, which beats the 0.812 from four descriptors alone.

Take away: a good training score on its own means nothing. The broken model had the best training score in the notebook.

Step 7. A different model (section 7)

A random forest on the same four descriptors, and five-fold cross-validation.

NOTE

You should see `forest R2 = 0.928` and a cross-validated mean near 0.884.

Step 8. The shape of chemical space (section 8)

PCA of the fingerprints, coloured by solubility.

NOTE

You should see a cloud of 1128 points where the colour changes smoothly across the map. That smoothness is the entire premise of the field: similar structure, similar property.

Step 9. Your own molecule (section 9)

Put any SMILES into `my_smiles` and predict its solubility with all three models.

Try it: a molecule from your own laboratory work, or from a bottle on your bench. Compare the three predictions. They will not agree, and the disagreement is informative.

If you finish early

Section 10 and the optional tail: save the model, then predict **toxicity** instead of solubility on a different dataset. The target is a label rather than a number, so the problem becomes classification. Expect about 0.958 accuracy.

Part 3. Agentic AI in chemistry

16:00, about fifteen minutes. Nothing to type.

What an agent is, what changes when its tools are chemistry codes, and what people have already built: ChemCrow, Coscientist, ChemGraph, and autonomous laboratories.

Part 4. Hands-on 2: an agent that does chemistry

16:15, about thirty-five minutes. Open `notebooks/02-agent-that-does-chemistry.ipynb`. You need your API key in `.env`.

Step 1. One plain call (section 1)

Ask a language model a question, then ask it for the exact molecular weight of paracetamol.

NOTE

You should see the model print a number. **Run that cell three times.** The answer may change, and it may be wrong in the last decimal. The model is recalling text, not computing. RDKit says `151.165` exactly.

Step 2. Three tools (section 2)

A tool is an ordinary Python function plus a description. You build three: exact properties, a geometry optimiser, and the solubility model you fitted before tea.

NOTE

You should see the three tool names, then a test of each. `molecule_properties` on paracetamol gives `MolWt 151.165`. An invalid SMILES returns a sentence rather than a crash, which is deliberate.

Step 3. The loop (section 3)

Wire the model to the tools and ask the same molecular-weight question again.

NOTE

You should see a trace: `[model asks for] molecule_properties(...)`, then `[tool returns ]`, then the answer. **The model did not answer from memory this time.** Compare it with step 1.

Step 4. Two tools in one request (section 4)

Ask it to build water, optimise the geometry and report the energy.

NOTE

You should see `optimise_geometry({'smiles': 'O'})` and a final energy of `1.8793 eV` after 3 steps. That number is a real calculation, not a recollection.

Then the solubility comparison between ethanol and naphthalene, where it calls the model **you** fitted an hour ago.

Step 5. Your turn (section 5)

Three suggested prompts, then write your own in the empty cell.

Try it: ask for something none of the tools can do, and watch what happens. A good agent says it cannot. A careless one invents an answer. Knowing which you have is the difference between a useful instrument and a confident liar.

Step 6. The research-grade version (demonstration)

Nothing to type. You watch **ChemGraph**, an agent framework from Argonne National Laboratory, take the same kind of sentence and drive a real calculation.

NOTE

You should see it build water, optimise the geometry and report **1.879 eV**, converged in 4 steps. That is the same number your own agent produced a few minutes earlier, from a research tool with thirty times the machinery.

The point is not that ChemGraph is better. It is that the thing you built in twenty minutes and the thing a national laboratory ships are the same three parts: a model, tools, and a loop.

Part 5. Where this goes

16:50. Self-driving laboratories, foundation models for matter, and what to learn next. The reading list is yours to take away.

If you get stuck at any point

Raise your hand. The most common problems are an environment that is not activated (`source .venv/bin/activate`), and a key that is not being read (check `.env` has no spaces around the `=`).

Everything from today, including the slides and the morning's MATLAB and Octave code, stays at github.com/Amrita-School-of-AI/ai-in-material-science .